

This assignment is concerned with differential kinematics of serial link robotic arms.

The following exercises are solved with Python and utilize Jupyter notebooks together with standard robotics libraries (NumPy, SpatialMath, and the Robotics Toolbox for Python). Some tasks involve the use of ROS 2 (Jazzy) on a Linux (Ubuntu) system for communicating with the *myCobot 280* and its software stack. Code is executed in the provided notebooks; follow the instructions in the notebook cells for each task.

Learning Objectives Differential Kinematics Lab

After completing the *Differential Kinematics* lab, students will have a comprehensive understanding of how joint velocities relate to the end-effectors linear and angular velocities in serial-link robotic manipulators. They will be able to derive, interpret, and apply the fundamental equations of differential kinematics to model and control robotic motion at the velocity level.

A primary objective of this lab is to understand the role of the *geometric Jacobian* in mapping joint space velocities $\dot{\mathbf{q}}$ to end-effector velocities $\boldsymbol{\nu}_e = [\boldsymbol{\omega}_e^T \dot{\mathbf{p}}_e^T]^T$. Students will learn how to construct the Jacobian matrix for both revolute and prismatic joints, interpret its physical meaning, and use it to describe the instantaneous motion of the robot end-effector in Cartesian space. They will gain insight into how the Jacobian structure depends on the manipulators configuration and how it changes dynamically as the robot moves.

Students will also explore important properties of the Jacobian, such as its rank, null space, and singularities. They will understand how singular configurations limit the robots motion capabilities and how redundancy introduces infinitely many joint velocity solutions for a given end-effector velocity. Furthermore, they will learn how to transform the Jacobian into different coordinate frames and apply it to tasks such as visual servoing or camera-based control.

Another key learning outcome is mastering the use of *inverse differential kinematics* to solve velocity-level control problems. Students will learn how to compute joint velocities from desired end-effector velocities using the Jacobian inverse and pseudo-inverse. They will implement *resolved-rate motion control* schemes to track desired Cartesian trajectories and understand how these methods integrate into real-time robot control.

Additionally, the lab introduces numerical techniques for solving differential equations related to robot motion. Students will implement Euler and Runge-Kutta methods to integrate velocity commands and generate joint trajectories. They will also learn how to extend resolved-rate control with adaptive Jacobian-based schemes that account for changes in the manipulator configuration.

By the end of this lab, students will be able to:

- Explain the concept of differential kinematics and its importance in robotic motion analysis and control.

- Derive and interpret the geometric Jacobian for robotic manipulators with revolute and prismatic joints.
- Analyze Jacobian properties such as rank, singularities, and null space, and understand their implications.
- Compute joint velocities from desired end-effector velocities using inverse differential kinematics.
- Implement resolved-rate motion control and track trajectories in Cartesian space.
- Apply numerical integration methods to generate joint trajectories from velocity commands.
- Transform Jacobians into alternative reference frames for tasks such as camera-based or end-effector control.

These skills form a critical foundation for advanced topics in robotics such as motion planning, redundancy resolution, dynamic control, and optimization-based trajectory generation. Mastery of differential kinematics enables students to bridge the gap between kinematic modeling and real-time robot control in practical applications.

Differential Kinematics

Forward kinematics describes the relationship between the joint configuration of a robot and the position and orientation of its end-effector. In many robotic applications, however, it is not sufficient to determine only the pose of the end-effector. For motion generation and control, it is also necessary to describe how the end-effector moves as the robot joints move. This relationship is the subject of *differential kinematics* [1].

Consider, for example, a robotic manipulator that has to move its end-effector along a specified Cartesian path with a desired velocity. While the desired motion is naturally specified in the workspace, the actuators of the robot control the individual joint velocities. It is therefore necessary to determine how the joint velocities have to be chosen in order to generate the desired translational and rotational velocity of the end-effector. Differential kinematics provides this relationship and forms an important basis for robot motion control, trajectory tracking, inverse kinematics, and the analysis of singular configurations.

For a manipulator with n joints, the joint configuration is described by

$$\mathbf{q} = [q_1 \quad q_2 \quad \cdots \quad q_n]^T. \quad (1)$$

The forward kinematics defines the end-effector pose as a nonlinear function of the joint configuration,

$${}^b\mathbf{T}_e(\mathbf{q}) = \begin{bmatrix} {}^b\mathbf{R}_e(\mathbf{q}) & {}^b\mathbf{t}_e(\mathbf{q}) \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (2)$$

Differential kinematics considers infinitesimal changes of this mapping and relates the joint velocity

$$\dot{\mathbf{q}} = [\dot{q}_1 \quad \dot{q}_2 \quad \cdots \quad \dot{q}_n]^T \quad (3)$$

to the instantaneous linear and angular velocity of the end-effector.

The end-effector velocity can be represented by the six-dimensional vector

$$\mathbf{v}_e = \begin{bmatrix} \dot{\mathbf{t}}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = [v_x \quad v_y \quad v_z \quad \omega_x \quad \omega_y \quad \omega_z]^T, \quad (4)$$

where $\dot{\mathbf{t}}_e$ denotes the translational velocity of the end-effector and $\boldsymbol{\omega}_e$ its angular velocity. The relationship between joint velocities and end-effector velocity is given by

$$\boxed{\mathbf{v}_e = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}} \quad (5)$$

where $\mathbf{J}(\mathbf{q})$ is the *manipulator Jacobian*. For an n -joint manipulator, the geometric Jacobian has the dimension

$$\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times n}. \quad (6)$$

It can be partitioned into a translational and a rotational part,

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{J}_v(\mathbf{q}) \\ \mathbf{J}_\omega(\mathbf{q}) \end{bmatrix}, \quad (7)$$

such that

$$\dot{\mathbf{t}}_e = \mathbf{J}_v(\mathbf{q})\dot{\mathbf{q}}, \quad \boldsymbol{\omega}_e = \mathbf{J}_\omega(\mathbf{q})\dot{\mathbf{q}}. \quad (8)$$

The Jacobian therefore describes how an instantaneous motion of each individual joint contributes to the translational and rotational motion of the end-effector. Unlike the forward kinematic transformation, the Jacobian depends on the current robot configuration. Consequently, the same joint velocities may produce different end-effector velocities at different configurations of the manipulator.

The relation

$$\mathbf{v}_e = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \quad (9)$$

describes the *forward differential kinematics*: given the current configuration and the joint velocities, the resulting end-effector velocity can be determined. The corresponding *inverse differential kinematics* considers the opposite problem. Given a desired end-effector velocity $\mathbf{v}_{e,d}$, suitable joint velocities have to be determined from

$$\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{v}_{e,d}. \quad (10)$$

For a square and nonsingular Jacobian, this relationship can locally be inverted,

$$\dot{\mathbf{q}} = \mathbf{J}^{-1}(\mathbf{q})\mathbf{v}_{e,d}. \quad (11)$$

For non-square or redundant manipulators, the Jacobian pseudoinverse can be used instead,

$$\dot{\mathbf{q}} = \mathbf{J}^\dagger(\mathbf{q})\mathbf{v}_{e,d}. \quad (12)$$

This formulation also provides the basis for exploiting redundant degrees of freedom for secondary objectives.

An important consequence of differential kinematics is the concept of *kinematic singularities*. At certain configurations, the Jacobian loses rank. In such a configuration, the manipulator loses the ability to generate motion in one or more Cartesian directions, and the inverse differential kinematic problem becomes ill-conditioned or cannot be solved uniquely. Analysis of the Jacobian therefore provides important information about the instantaneous motion capabilities of a robotic manipulator.

Differential kinematics establishes the connection between the pose-level description introduced by forward kinematics and the velocity-level description required for robot control. The following sections derive the manipulator Jacobian and investigate its use for forward and inverse differential kinematics as well as for the identification of singular robot configurations.

Differential Kinematics

Consider the forward kinematics of a robot arm described by the homogeneous transformation

$$\mathbf{T}_e(\mathbf{q}) = \begin{bmatrix} \mathbf{R}_e(\mathbf{q}) & \mathbf{p}_e(\mathbf{q}) \\ \mathbf{0} & 1 \end{bmatrix}, \quad (1)$$

where $\mathbf{q} = [q_1, \dots, q_n]^T$ is the vector of joint variables. Differential kinematics is concerned with the problem to determine the translational ($\dot{\mathbf{p}}_e$) and angular ($\boldsymbol{\omega}_e$) velocity of the end effector given the joint positions $\mathbf{q}$ and velocities $\dot{\mathbf{q}}$. The goal is to express the end effector linear velocity $\dot{\mathbf{p}}_e$ and angular velocity $\boldsymbol{\omega}_e$ as a function of the joint velocities $\dot{\mathbf{q}}$.

$$\dot{\mathbf{p}}_e = \mathbf{J}_p(\mathbf{q})\dot{\mathbf{q}} \quad (2)$$

$$\boldsymbol{\omega}_e = \mathbf{J}_o(\mathbf{q})\dot{\mathbf{q}} \quad (3)$$

$\mathbf{J}_p(\mathbf{q})$ is the $3 \times n$ matrix that describes the effect of the joint velocity $\dot{\mathbf{q}}$ on the end effector linear velocity $\dot{\mathbf{p}}_e$. $\mathbf{J}_o(\mathbf{q})$ is the $3 \times n$ matrix that describes the effect of the joint velocity $\dot{\mathbf{q}}$ on end effector angular velocity $\boldsymbol{\omega}_e$.

In a compact representation the linear and angular velocity are combined into the 6×1 -vector of total velocity $\boldsymbol{\nu}_e$. The relationship is provided by the $6 \times n$ -Jacobian matrix $\mathbf{J}(\mathbf{q})$ which depends on the joint configuration $(q_1, \dots, q_n)$.

$$\boldsymbol{\nu}_e = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \quad (4)$$

The matrix $\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{J}_p(\mathbf{q}) \\ \mathbf{J}_o(\mathbf{q}) \end{bmatrix}$ denotes the manipulators **geometric** Jacobian, which depends on the joint configuration $\mathbf{q}$.

For prismatic joints the geometric Jacobian is defined by

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{J}_p(\mathbf{q}) \\ \mathbf{J}_o(\mathbf{q}) \end{bmatrix} = \begin{bmatrix} \mathbf{z}_{i-1} \\ \mathbf{0} \end{bmatrix} \quad (5)$$

For revolute joints the geometric Jacobian is defined by

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{J}_p(\mathbf{q}) \\ \mathbf{J}_o(\mathbf{q}) \end{bmatrix} = \begin{bmatrix} \mathbf{z}_{i-1} \times (\mathbf{p}_e - \mathbf{p}_{i-1}) \\ \mathbf{z}_i \end{bmatrix} \quad (6)$$

in which $\mathbf{z}_i$ denotes the i -th joint axis, $\mathbf{p}_e$ the translation vector of the end effector frame and $\mathbf{p}_i$ the vector to the origin of the i -th joint frame. Notice, that $\mathbf{z}_i$, $\mathbf{p}_e$, $\mathbf{p}_i$ are extracted from the homogeneous transforms of joint frames $\mathbf{H}_i$ and end effector frame $\mathbf{H}_e$. In particular $\mathbf{z}_i = \mathbf{H}_i(1 : 3, 3)$ is given by the third column of the rotation matrix part of $\mathbf{H}_i$, $\mathbf{p}_i = \mathbf{H}_i(1 : 3, 4)$ is given by the first three elements of the fourth column of $\mathbf{H}_i$, $\mathbf{p}_e = \mathbf{H}_e(1 : 3, 4)$ is given by the first three elements of the fourth column of $\mathbf{H}_e$.

Be aware that the Robotics Toolbox for Python (e.g., `robot.jacob0(q)` and `robot.jacobe(q)`) returns the Jacobian consistent with the spatial velocity ordering

$$\boldsymbol{\nu} = \begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{p}_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}. \quad (7)$$

When comparing results across conventions, ensure the ordering of translational and rotational components is consistent.

Jacobian Calculation with SymPy

In Python, SymPy can be used to compute Jacobians symbolically. The Jacobian of a vector function is a matrix of the partial derivatives of that function. The following code computes the Jacobian matrix of the vector function $[xyz, y^2, x + z]$ with respect to the variables $[x, y, z]$.

```
import sympy as sp

x, y, z = sp.symbols('x y z')
f = sp.Matrix([x*y*z, y**2, x + z])
v = sp.Matrix([x, y, z])

J = f.jacobian(v)
J

Matrix([
[ y*z,  x*z,  x*y],
[  0,  2*y,   0],
[  1,   0,   1]
])
```

Fig. 1 shows a two link planar arm. The forward kinematics of the end effector $\{x_2, y_2, z_2\}$ w.r.t. robot base frame $\{x_0, y_0, z_0\}$ is given by $\mathbf{T}_2^0(\mathbf{q})$:

$$\mathbf{T}_2^0(\mathbf{q}) = \mathbf{A}_1^0(\mathbf{q})\mathbf{A}_2^1(\mathbf{q}) = \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 & a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 & a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

with

$$\mathbf{A}_i^{i-1}(\mathbf{q}) = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i & 0 & a_i \sin \theta_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad i = 1, 2$$

Now, please complete tasks 1 to 3 in the template.

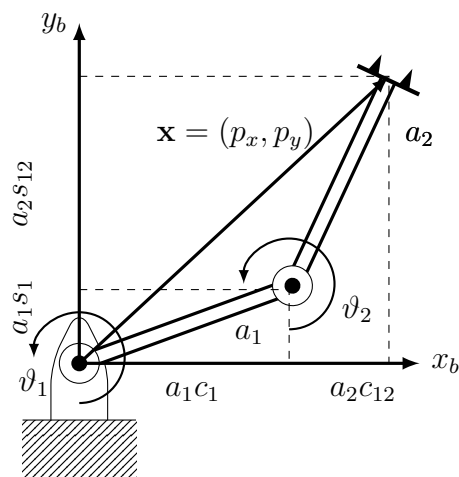


Figure 1: Kinematic structure two link planar arm

Geometric Jacobian with Robotics Toolbox for Python

The Robotics Toolbox for Python provides functionalities for differential kinematics, in particular the calculation of the geometric Jacobian of arbitrary frames w.r.t. the robot base frame. For a given robot model and joint configuration, the command

```
J = robot.jacob0(q)
```

computes the geometric Jacobian expressed in the base frame for the end-effector. Alternatively,

```
J_E = robot.jacobe(q)
```

computes the geometric Jacobian expressed in the end-effector frame.

The resulting 6-by-n matrix is the geometric Jacobian of the end effector with the given configuration, where n is the number of degrees of freedom for the end effector. The Jacobian maps the joint-space velocity to the end effector spatial velocity. The end effector velocity equals:

$$\begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{p}_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \quad (8)$$

in which $\boldsymbol{\omega}$ denotes the angular velocity and $\dot{\mathbf{p}}$ the translational velocity of the end effector frame (or any other frame specified by the robot model). $\mathbf{q} = [q_1, \dots, q_n]^T$ denotes the joint configuration and $\dot{\mathbf{q}}$ the joint-space velocity.

A kinematic singularity is a configuration within the robot's workspace where the Jacobian loses rank. Intuitively, this means that at a kinematic singularity the robot loses its ability to move the end effector in some direction no matter how the joints are actuated. Kinematic singularities often emerge when the robot's joints or links line up.

Now, please complete tasks 4 to 7 in the template.

The Jacobian matrix relates differential joint coordinate motion to differential Cartesian motion $d\mathbf{x} = \mathbf{J}(\mathbf{q}) d\mathbf{q}$. The following tasks are based on the 6-DOF *myCobot 280* robot arm with a 6×6 -Jacobian.

The geometric Jacobian is usually defined for translational and angular velocities expressed w.r.t. the base frame. In applications such as visual servoing one would like to express translational and angular velocities expressed w.r.t. a camera or end effector frame. If it is desired to represent the Jacobian in a different frame u , it is sufficient to know the relative rotation matrix $\mathbf{R}^u$. The relationship between velocities in the base frame and frame u is

$$\begin{bmatrix} \dot{\mathbf{p}}_e^u \\ \boldsymbol{\omega}_e^u \end{bmatrix} = \begin{bmatrix} \mathbf{R}^u & \mathbf{0} \\ \mathbf{0} & \mathbf{R}^u \end{bmatrix} \begin{bmatrix} \dot{\mathbf{p}}^e \\ \boldsymbol{\omega}^e \end{bmatrix} \quad (9)$$

which gives

$$\mathbf{J}^u = \begin{bmatrix} \mathbf{R}^u & \mathbf{0} \\ \mathbf{0} & \mathbf{R}^u \end{bmatrix} \mathbf{J} \quad (10)$$

Now, please complete tasks 8 to 10 in the template.

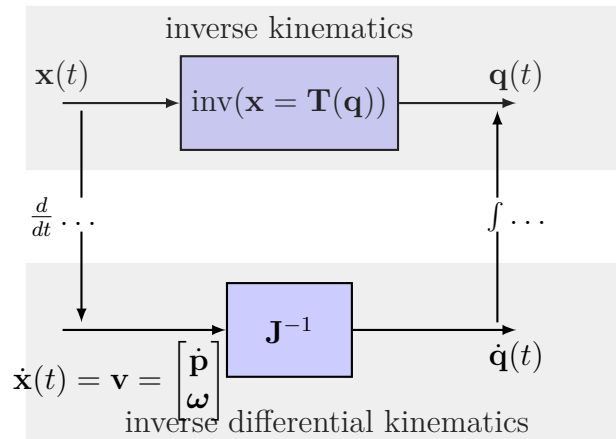


Figure 2: Resolved rate motion control with differential kinematics

Resolved Rate Motion Control

The Jacobian inverse technique is a simple yet effective way of implementing inverse kinematics. Resolved rate motion constitutes an open loop control technique to track a desired end effector motion in task space $\dot{\mathbf{x}}_d(t)$. It rests upon the numerical integration of inverse differential kinematics

$$\dot{\mathbf{q}} = \mathbf{J}(\mathbf{q})^{-1} \dot{\mathbf{x}}, \quad (11)$$

yielding

$$\mathbf{q}(t) = \int_{\tau=0}^t \mathbf{J}(\mathbf{q}(\tau))^{-1} \dot{\mathbf{x}}(\tau) d\tau + \mathbf{q}(0) \quad (12)$$

The integral is approximated by numerical integration with the Euler method by the equivalent difference equation

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \mathbf{J}(\mathbf{q}_k)^{-1} \dot{\mathbf{x}}_k \Delta t, \quad (13)$$

at discrete time steps $t_k = k\Delta t$ with short hand notation of joint configuration $\mathbf{q}_k = \mathbf{q}(t_k)$ and task space pose $\dot{\mathbf{x}}_k = \dot{\mathbf{x}}(t_k)$. The numerical integration in (13) solves the motion control task in joint space. Figure 2 illustrates the open loop resolved rate motion control scheme. Numerical integration of (11) is accomplished with Euler or Runge-Kutta methods. Figure 2 illustrates the open loop resolved rate motion control scheme.

Euler Method for Ordinary Differential Equations (ODE)

Closed form analytical solutions of nonlinear ODE are often unavailable. Numerical integration approximates the solution of ODEs numerically with an accuracy that is usually sufficient for applications such as the simulation of dynamic systems. Without loss of generality, a first order differential equation of the form

$$\dot{x}(t) = f(t, x(t)), \quad x(t_0) = x_0 \quad (14)$$

is considered in which time t denotes the independent variable. The condition $x(t_0) = x_0$ denotes the initial value of the function $x(t)$. These type of *initial value problems* are solved numerically using *linear multi-step methods* or *Runge-Kutta methods*.

By replacing the derivative in equation 14 by the finite difference approximation, one obtains:

$$\dot{x}(t) \simeq \frac{x(t + \Delta t) - x(t)}{\Delta t} \quad (15)$$

where Δt denotes a step size, by rearranging and choosing a step value and setting: $t_n = t_0 + n\Delta t$ and $x_n = x(t_n)$, yields that one step of the Euler method from t_n to t_{n+1} is

$$x_{n+1} \simeq x_n + f(t_n, x_n) \Delta t \quad (16)$$

The smaller the step size Δt the more accurate the approximation, however resulting in a higher number of iterations and an increase in the computational effort.

Assignment Resolved Rate Motion Control

Now, please complete tasks 11 to 19 in the template.

Notes:

- Figure 3 for tasks 14, 16,17

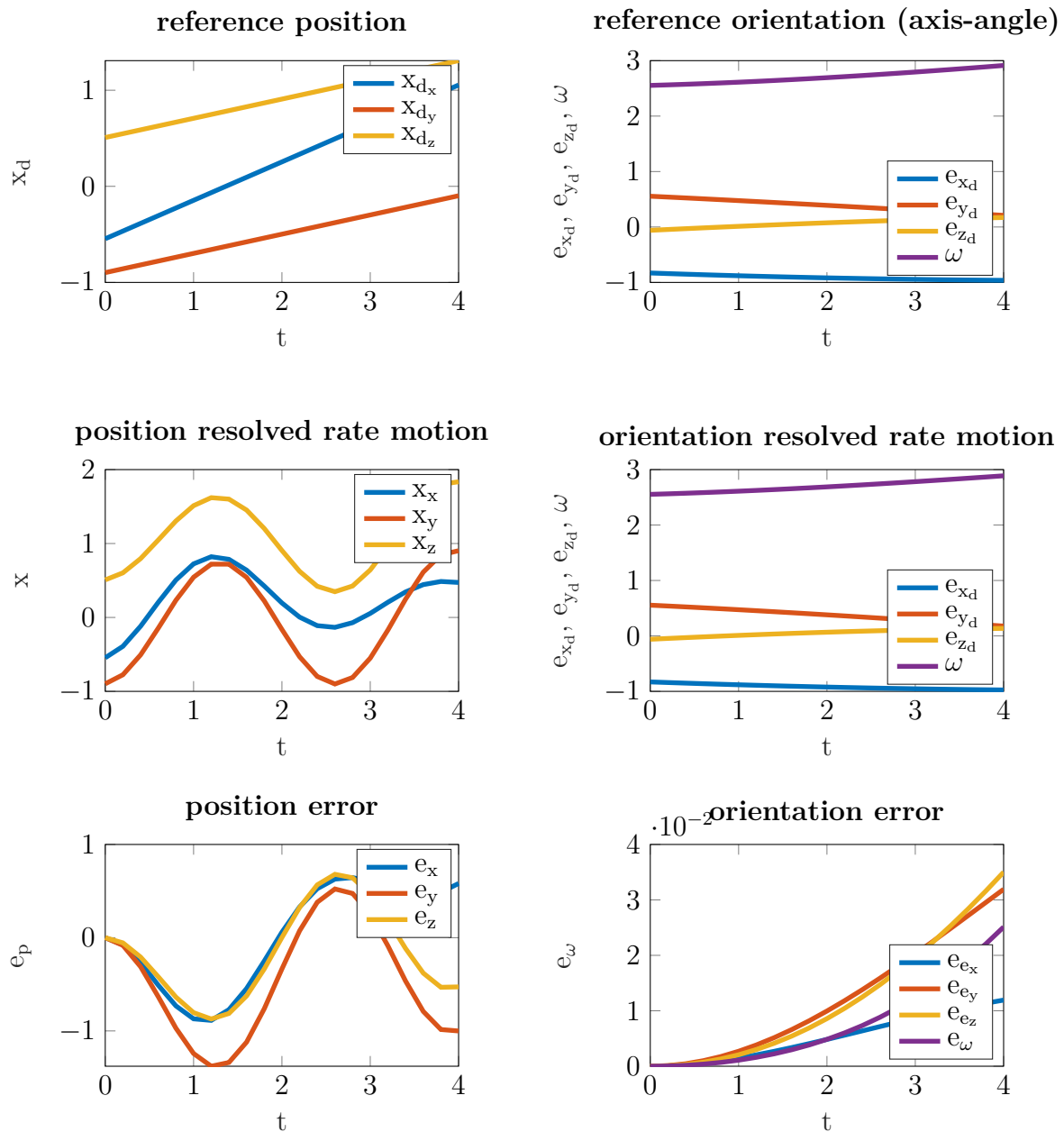


Figure 3: Reference trajectory in task space

Advanced Resolved Rate Motion

A more advanced resolved rate motion scheme (open-loop) considers the variation of the Jacobian with the pose and relies on the integration.

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \mathbf{J}_w(\mathbf{q}_k)^{-1} \dot{\mathbf{x}}_d \Delta t \quad (17)$$

The key difference is that this scheme employs the Jacobian $\mathbf{J}_w(\mathbf{q}_k)$ of the current joint configuration $\mathbf{q}_k$

Now, please complete tasks 20 to 23 in the template.

Competences

After successfully completing the Differential Kinematics lab, students will have acquired the following competences:

- They understand the concept of differential kinematics and its significance for analyzing and controlling robotic motion at the velocity level.
- They are able to derive, interpret, and apply the fundamental equations relating joint velocities to the end-effectors linear and angular velocities in serial-link manipulators.
- They can construct the geometric Jacobian for manipulators with revolute and prismatic joints, interpret its physical meaning, and use it to describe instantaneous motion in Cartesian space.
- They are capable of analyzing important Jacobian properties such as rank, singularities, null space, and redundancy, and understand their implications for robot mobility and control.
- They can compute joint velocities from desired end-effector velocities using inverse differential kinematics, including the use of Jacobian inverses and pseudo-inverses.
- They are proficient in implementing resolved-rate motion control schemes to track Cartesian trajectories and understand how these methods integrate into real-time robot control systems.
- They can apply numerical integration methods, such as Euler and Runge-Kutta, to generate joint trajectories from velocity commands and simulate robotic motion over time.
- They are able to transform Jacobians into alternative reference frames and apply them to tasks such as camera-based control or visual servoing.
- They understand the occurrence and impact of singular configurations, and can devise strategies to handle redundancy and infinite solution spaces in velocity-level control.

- They have the skills to integrate differential kinematics into advanced robotics tasks, such as motion planning, optimization-based trajectory generation, and dynamic control.

References

- [1] Bruno Siciliano, Luigi Villani, Giuseppe Oriolo, and Alessandro De Luca. *Foundations of Robotics*. Advanced Textbooks in Control and Signal Processing. Springer Nature Switzerland, Cham, 2025.